

【工具】NVM完全指南：Node.js版本管理工具的安装与使用详解

原创 No厉害 合木子NVer 2025年07月31日 08:00 山东

前言

在现代前端和后端开发中，Node.js已经成为不可或缺的运行环境。然而，不同的项目往往需要不同版本的Node.js，这就产生了版本管理的需求。NVM (Node.js Version Manager) 作为最受欢迎的Node.js版本管理工具，能够帮助开发者在同一台机器上安装、管理和切换多个Node.js版本。本文将详细介绍NVM的安装、配置和使用方法。

一、NVM简介

(一) 什么是NVM

NVM全称为Node.js Version Management，是一个Node.js的版本管理工具。它允许开发者：

- 安装多个Node.js版本**：在同一台机器上安装不同版本的Node.js
- 快速切换版本**：根据项目需求快速切换Node.js版本
- 隔离项目环境**：不同项目使用独立的Node.js环境
- 解决兼容性问题**：避免因Node.js版本不兼容导致的项目问题

(二) NVM的优势

1. 版本隔离

```
# 项目A使用Node.js 14.x
cd project-a
nvm use 14.21.3

# 项目B使用Node.js 18.x
cd project-b
nvm use 18.19.0
```

2. 简化管理

- 一条命令安装Node.js版本
- 一条命令切换版本

- 自动管理npm版本

3. 团队协作

- 确保团队成员使用相同的Node.js版本
- 通过.nvmrc文件统一项目环境

(三) NVM vs 其他工具

工具	平台支持	特点	适用场景
nvm	Windows/macOS/Linux	功能全面，社区活跃	通用版本管理
n	macOS/Linux	轻量级，简单易用	简单版本切换
fnm	Windows/macOS/Linux	速度快，Rust编写	追求性能
volta	Windows/macOS/Linux	项目级管理	企业级项目

二、Windows系统安装NVM

(一) 准备工作

1. 卸载现有Node.js

```
# 1. 通过控制面板卸载Node.js
# 2. 删除残留文件夹
Remove-Item -Recurse -Force "C:\Program Files\nodejs"
Remove-Item -Recurse -Force "C:\Users\{用户名}\AppData\Roaming\npm"

# 3. 清理环境变量中的Node.js路径
```

2. 下载NVM for Windows

- NVM 官网下载地址 <https://nvm.uihtm.com/>
- 官方GitHub地址 <https://github.com/coreybutler/nvm-windows/releases>
- 当前最新版本：v1.2.2（更新日期：2025-01-01）
- 下载文件：nvm-setup.zip

- 注意，nvm 1.2.2版本，安装node低于14的版本有无法安装的问题

```
选择管理员: 命令提示符
nvm --version : Displays the current running version of nvm for Windows. Aliased as v.

C:\Users\Administrator>nvm -v
1.2.2

C:\Users\Administrator>nvm i 12.22.2
Downloading node.js version 12.22.2 (64-bit)...
Complete
Downloading npm...
Creating C:\Users\ADMINI~1\AppData\Local\Temp\2\nvm-install1-1084875866\temp
Downloading npm version 6.14.13... Complete
Installing npm v6.14.13...
error installing 12.22.2: open C:\Users\ADMINI~1\AppData\Local\Temp\2\nvm-npm-595727315\npm-v6.14.13.zip: The system cannot find the file specified.
C:\Users\Administrator>
```

公众号 · 合木子NWer

(二) 安装步骤

1. 运行安装程序

```
# 解压下载的nvm-setup.zip
# 运行nvm-setup.exe
```

2. 选择安装路径

- **NVM安装路径**：建议使用默认路径或选择较短路径（如C:\nvm）
- **Node.js路径**：选择Node.js的符号链接路径（如C:\nodejs）

3. 完成安装

安装完成后，打开新的命令提示符或PowerShell窗口，输入以下命令验证：

```
nvm version
# 应该显示：1.2.2
```

(三) 配置国内镜像

为了提高下载速度，建议配置国内镜像源：

1. 方法一：命令行配置

```
# 阿里云镜像
nvm node_mirror https://npmmirror.com/mirrors/node/
nvm npm_mirror https://npmmirror.com/mirrors/npm/

# 腾讯云镜像
```

```
nvm node_mirror http://mirrors.cloud.tencent.com/nodejs-release/  
nvm npm_mirror http://mirrors.cloud.tencent.com/npm/
```

2. 方法二：修改配置文件

编辑NVM安装目录下的 `settings.txt` 文件：

```
root: C:\nvm  
path: C:\nodejs  
node_mirror: https://npmmirror.com/mirrors/node/  
npm_mirror: https://npmmirror.com/mirrors/npm/
```

三、macOS系统安装NVM

(一) 安装前准备

1. 确保已安装Git

```
# 检查Git是否已安装  
git --version  
  
# 如果未安装，通过Xcode Command Line Tools安装  
xcode-select --install
```

2. 检查Shell配置文件

```
# 进入用户主目录  
cd ~  
  
# 查看现有配置文件  
ls -a  
  
# 如果没有.bash_profile，创建一个  
touch ~/.bash_profile
```

(二) 安装NVM

1. 使用curl安装

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
```

2. 使用wget安装

```
wget -qO- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
```

3. 重新加载配置

```
# 重新加载.bash_profile
source ~/.bash_profile

# 或者重启终端
```

4. 验证安装

```
nvm --version

# 应该显示版本号，如：0.39.3
```

(三) 注意事项

1. 不要使用Homebrew安装

```
# ❌ 错误做法
brew install nvm

# ✅ 正确做法：使用官方安装脚本
```

2. 环境变量自动配置

安装脚本会自动在 `.bash_profile` 中添加以下内容：

```
export NVM_DIR="$HOME/.nvm"

[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"# This Loads nvm

[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"# This Loads nvm bash_completion
```

四、Linux系統安裝NVM

(一) 下載安裝

1. 下載壓縮包

```
cd /  
wget https://github.com/nvm-sh/nvm/archive/refs/tags/v0.39.1.tar.gz
```

2. 解壓安裝

```
# 創建nvm目錄  
mkdir -p ~/.nvm  
  
# 解壓到nvm目錄  
tar -zxvf nvm-0.39.1.tar.gz -C ~/.nvm
```

3. 配置環境變量

```
# 編輯bashrc文件  
vim ~/.bashrc  
  
# 在文件末尾添加以下內容  
export NVM_DIR="$HOME/.nvm/nvm-0.39.1"  
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"# This loads nvm  
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"# This loads nvm bash completion
```

4. 使配置生效

```
source ~/.bashrc
```

(二) 驗證安裝

```
nvm --version  
# 顯示版本号表示安裝成功
```

五、NVM常用命令详解

(一) 基础命令

1. 查看版本和帮助

```
nvm version          # 显示nvm版本
nvm --help# 显示帮助信息
```

2. 架构相关

```
nvm arch              # 显示当前系统架构（32位或64位）
```

(二) Node.js安装管理

1. 查看可用版本

```
nvm list available  # 显示可安装的Node.js版本列表
nvm ls-remote       # (macOS/Linux) 显示远程可用版本
```

2. 安装Node.js

```
nvm install latest      # 安装最新版本
nvm install 18.19.0     # 安装指定版本
nvm install 18.19.0 32  # 安装32位版本（Windows）
nvm install --lts       # 安装最新LTS版本
```

3. 查看已安装版本

```
nvm list              # 显示已安装的版本
nvm ls                # 简写形式
```

4. 使用和切换版本

```
nvm use 18.19.0    # 切换到指定版本
nvm use latest     # 切换到最新版本
nvm use --lts      # 切换到最新LTS版本
```

5. 卸载版本

```
nvm uninstall 16.20.0 # 卸载指定版本
```

(三) 配置管理

1. 设置默认版本

```
nvm alias default 18.19.0 # 设置默认版本
nvm alias default node    # 设置最新版本为默认
```

2. 镜像配置

```
nvm node_mirror https://npmmirror.com/mirrors/node/
nvm npm_mirror  https://npmmirror.com/mirrors/npm/
```

3. 代理设置

```
nvm proxy http://proxy.company.com:8080 # 设置代理
nvm proxy none                          # 移除代理
```

(四) 高级功能

1. 目录管理

```
nvm root          # 显示nvm根目录
nvm root C:\new-nvm-path # 设置新的根目录
```

2. 开关控制


```
nvm on          # 启用nvm
nvm off         # 禁用nvm
```

六、实际使用场景

(一) 项目级版本管理

1. 创建.nvmrc文件

```
# 在项目根目录创建.nvmrc文件
echo"18.19.0" > .nvmrc
```

2. 自动切换版本

```
# 进入项目目录时自动切换
nvm use
# 如果版本未安装，先安装
nvm install
```

(二) 多项目环境切换

1. 前端项目（需要Node.js 18）

```
cd frontend-project
nvm use 18.19.0
npm install
npm run dev
```

2. 后端项目（需要Node.js 16）

```
cd backend-project
nvm use 16.20.0
npm install
npm start
```

(三) 团队协作

1. 统一开发环境

```
// package.json
{
  "engines": {
    "node": ">=18.19.0 <19.0.0",
    "npm": ">=9.0.0"
  }
}
```

2. CI/CD配置

```
# .github/workflows/ci.yml
-name: UseNode.js
uses: actions/setup-node@v3
with:
  node-version-file: '.nvmrc'
```

七、常见问题与解决方案

(一) 安装问题

1. 权限错误

```
# Windows: 以管理员身份运行
# macOS/Linux: 检查文件权限
chmod +x ~/.nvm/nvm.sh
```

2. 网络问题

```
# 配置代理或使用镜像源
nvm node_mirror https://npmmirror.com/mirrors/node/
```

(二) 使用问题

1. 命令未找到

```
# 重新加载配置文件
source ~/.bashrc
source ~/.bash_profile
```

2. 版本切换失效

```
# 检查环境变量
echo $PATH

# 确保nvm路径在最前面
```

(三) 性能优化

1. 清理缓存

```
# 清理npm缓存
npm cache clean --force

# 清理nvm缓存（删除临时文件）
```

2. 使用本地镜像

```
# 配置更快的镜像源
nvm node_mirror https://npmmirror.com/mirrors/node/
```

八、最佳实践

(一) 版本选择策略

1. 项目版本规划

- **新项目**：使用最新LTS版本
- **维护项目**：保持现有版本，谨慎升级
- **实验项目**：可以尝试最新版本

2. 版本命名规范

```
# 使用语义化版本号
nvm install 18.19.0 # 明确的版本号
nvm alias stable 18.19.0 # 为稳定版本创建别名
```

(二) 团队协作规范

1. 文档化要求

```
## 环境要求
- Node.js: 18.19.0 (使用nvm管理)
- npm: 9.6.0

## 快速开始
```bash
nvm install 18.19.0
nvm use 18.19.0
npm install
```

### 2. 自动化脚本

```
#!/bin/bash
setup.sh
if [-f ".nvmrc"]; then
 nvm use
if [$? -ne 0]; then
 nvm install
fi
fi
npm install
```

## (三) 维护建议

### 1. 定期清理

```
查看已安装版本
nvm list

卸载不再使用的版本
nvm uninstall 14.21.3
```

## 2. 备份配置

```
备份nvm配置
cp ~/.nvm/alias/default ~/.nvm/alias/default.backup
```

## 九、总结

NVM作为Node.js版本管理的标准工具，为开发者提供了强大而灵活的版本管理能力。通过本文的详细介绍，您应该能够：

### （一）掌握的技能

- 安装配置**：在不同操作系统上正确安装和配置NVM
- 版本管理**：熟练使用NVM命令管理Node.js版本
- 项目应用**：在实际项目中应用版本管理策略
- 问题解决**：处理常见的安装和使用问题

### （二）关键点

- 选择合适版本**：根据项目需求选择合适的NVM版本
- 配置镜像源**：使用国内镜像提高下载速度
- 团队协作**：建立统一的版本管理规范
- 持续维护**：定期清理和更新版本

### （三）发展趋势

随着Node.js生态的不断发展，版本管理工具也在持续演进。NVM作为最成熟的解决方案，将继续在开发者工具链中发挥重要作用。掌握NVM的使用，不仅能提高开发效率，更是现代Node.js开发的必备技能。

记住，好的工具配合正确的使用方法，才能发挥最大的价值。希望本文能帮助您更好地使用NVM，提升您的开发体验！

## 参考资料

- NVM官方文档
- NVM for Windows
- Node.js官方网站
- NVM中文文档



No厉害

喜欢作者

工具 · 目录

上一篇

【工具】jsDelivr CDN完全指南：免费高速的  
开源项目CDN服务

下一篇

【工具】conda环境管理工具